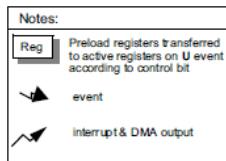
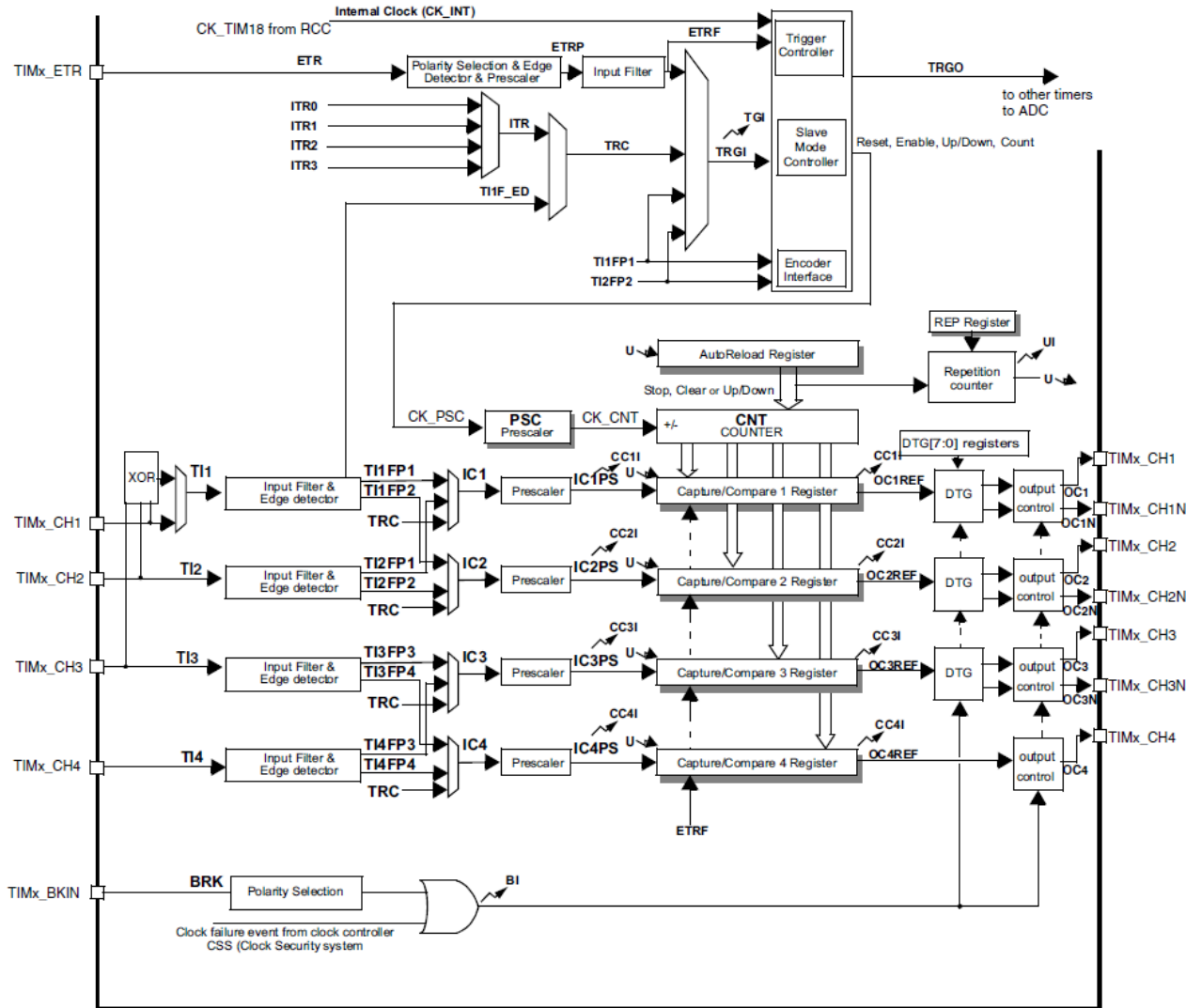




# Timer

- A peripheral based on a counter fed by the system clock
- Used to create periodical (or not) events
- Can generate IRQs at specific time intervals
- Can produce periodical signals useful for control systems (PWM)



# STM32 TIMER



# Lab Exercise:

- Use the HW Timer1 to generate a 20kHz interrupt signal
  - Switch on the Timer1
  - Activate the Interrupt source
  - Create the Interrupt Service Routine (ISR)
  - Toggle the LED into the ISR
  - Verify 10kHz on the header





# Lab Exercise, directions:

- Activate one timer channel in “Output Compare No Output” to use as a time-base
- In the Timer1, NVIC Settings tab of STM32CubeMX activate the TIM1 update interrupt, and ISR will be created in `stm32f4xx_it.c` (TIM1\_UP\_TIM10\_IRQHandler)
- To start the timer, call `HAL_TIM_Base_Start_IT(&htim1);`
- Place the toggle code into the ISR



# Lab Exercise:

- Use the HW Timer1 to generate a 20kHz PWM signal
  - Configure Timer1 channel 1 to generate a PWM output
  - Configure it initially for a 20% duty cycle
- Read the button into the ISR and increase the duty cycle when the button is pressed (2 seconds from 0 to 100%, then wrap around)

# Lab Exercise, directions:

- Activate the timer channel 1 in “PWM Generation CH1” to use as a PWM output
- Set the duty in the “Pulse” field of PWM channel1 tab, in Parameter Settings of TIM1
- To start the timer, call  
`HAL_TIM_Base_Start_IT(&htim1);`
- To start the PWM output call subsequently  
`HAL_TIM_PWM_Start(&htim1,TIM_CHANNEL_1);`
- To modify the duty, write **`htim1.Instance->CCR1`**



# Lab Exercise, directions:

- The interrupt is called 20k times/sec, while the duty cycle can span 0-4200
- You need to downsample the ISR calls:
  - Use the Repetition Counter setting into the Parameter Settings of TIM1 (N-1)

